UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA EN CIENCIAS Y SISTEMAS
ESTRUCTURA DE DATOS
PROYECTO 1 - FASE 2
PRIMER SEMESTRE 2026
Andy Jefferson González Fuentes 202231338

MANUAL TÉCNICO

Arquitectura del sistema

Capa	Clases	Responsabilidad
Entidad	Product	Estructura de datos del dominio
Estructuras	LinkedList, HashTable, AVLTree, BTree, BPlusTree	Indexado y búsqueda
Servicios	Logger, CSVLoader	I/O y validaciones
Control	Catalog	Coordinación e inserción atómica
Interfaz	Menu, Benchmark	Interacción con el usuario

La clase Catalog contiene toda operación de inserción, búsqueda o eliminación, que a su vez delega a cada estructura y garantiza la consistencia entre ellas.

Product

Estructura central del sistema. Todos los punteros en las estructuras apuntan a instancias de Product.

Campo	Tipo	Descripción	Clave en
id_sucursal	string	Sucursal del producto	-
name	string	Nombre del producto	AVL, Linkedist
barcode	string	Codigo de barra (único)	HashTable
category	string	Categoría del producto	BPlusTree
expiry_date	string	Fecha YYYY-MM-DD	BTree
brand	string	Marca del producto	-
price	double	Precio (> 0)	-
stock	int	Unidades disponibles (>= 0)	-

(DISEÑO DE LAS ESTRUCTURAS UTILIZADAS)

1. **LinkedList** - lista enlazada no ordenada

Inserción siempre al **frente** en O(1). Búsqueda secuencial nodo a nodo. Su propósito principal es servir de base para las pruebas comparativas del benchmark y proveer búsqueda por barcode.

2. **Tabla Hash** - inserciones de códigos de barras

La Tabla Hash implementada utiliza encadenamiento separado (separate chaining) como estrategia de resolución de colisiones. Cada posición del arreglo principal (_buckets) contiene un puntero al inicio de una lista enlazada de nodos HTNode, donde cada nodo almacena la clave (código de barras), un puntero al producto y un puntero al siguiente nodo en la cadena.

Parámetros de diseño:

- Tamaño de la tabla: HT_SIZE = 1024 buckets
- Función hash: algoritmo djb2 — hash = hash * 33 + c para cada carácter
- Resolución de colisiones: encadenamiento con inserción al inicio de la lista
- Clave de indexación: código de barras del producto (string)

Función hash djb2:

```
unsigned long hash = 5381;
for (char c : barcode)
hash = ((hash << 5) + hash) + (unsigned char)c;
return (int)(hash % HT_SIZE);
```

3. **AVLTree** - árbol AVL por nombre

Árbol binario de búsqueda que garantiza |factor_balance| <= 1 en cada nodo mediante rotaciones automáticas. La altura máxima es O(log n), lo que garantiza busqueda, insercion y eliminacion logaritmicas en todos los casos.

El factor de balance se calcula como altura(izq) - altura(der). Hay cuatro casos de desbalance que se corrigen con rotaciones:

Caso	Condición	Corrección
LL	balance > 1 y hijo izq pesado a la izq	Rotación simple derecha
LR	balance > 1 y hijo izq pesado a la der	Rotación doble izq-der
RR	balance < -1 y hijo der pesado a la der	Rotación simple izquierda
RL	balance < -1 y hijo der pesado a la izq	Rotación doble der-izq

4. **BTree** - arbol B por fecha de caducidad

Árbol B de grado mínimo t=4. Cada nodo tiene entre t-1=3 y 2t-1=7 claves. La raíz puede tener entre 1 y 7 claves. Con n=1000 la altura máxima es 5 niveles.

Justificación de t=4: maximiza la reducción de altura antes de entrar en la zona de rendimientos decrecientes. De t=2 a t=4 la altura baja de 10 a 5 niveles para n=1000. De t=4 en adelante, cada incremento reduce la altura a menos de 1 nivel adicional.

La clave expiry_date en formato **YYYY-MM-DD** es comparada como string, lo que produce el orden cronológico correcto sin conversión a fecha.

Casos de eliminación:

- A1: nodo hoja - eliminación directa desplazando claves
- A2a: nodo interno, hijo izquierdo tiene >= t claves - reemplazar con predecesor
- A2b: nodo interno, hijo derecho tiene >= t claves - reemplazar con sucesor
- A2c: ambos hijos tienen t-1 claves - merge de los dos hijos
- B: clave no está en el nodo - garantizar >= t claves antes de descender (redistribución o merge)

5. **Grafos** - representación de sucursales

El grafo representa la red de sucursales del sistema. Está implementado como un grafo dirigido con pesos duales (tiempo y costo) usando listas de adyacencia construidas completamente desde cero.

Componentes principales:

- GraphNode: nodo que representa una sucursal, con su ID y una lista enlazada de aristas salientes
- Edge: arista que almacena el ID del destino, el peso de tiempo y el peso de costo
- _nodes: lista enlazada de todos los GraphNode del grafo
- _count: número de nodos activos

Algoritmo de ruta más corta Dijkstra:

La implementación de Dijkstra no utiliza priority_queue de STL. En su lugar usa un arreglo de distancias dist[] y selecciona el nodo de menor distancia mediante un loop lineal en cada iteración. Soporta criterio dinámico: por tiempo o por costo según el parámetro useCost.

```
for (int iter = 0; iter < n; ++iter) {
    int u = nodo no visitado con menor dist[u];
    visited[u] = true;
    for cada arista e de u:
        if (dist[u] + w < dist[v]) actualizar dist[v] y prev[v];
}
```

6. **Colas**.

La cola está implementada como una clase genérica Queue<T> usando una lista enlazada con dos punteros: _head (frente) y _tail (final). Esto permite enqueue y dequeue en O(1) sin necesidad de arreglos redimensionables.

Casos de uso en el sistema:

- Cola de Ingreso (QueueType::INGRESO): productos que llegan a una sucursal esperando ser procesados
- Cola de Preparación (QueueType::PREPARACION): productos siendo empaquetados/preparados para traslado
- Cola de Salida (QueueType::SALIDA): productos listos para despacho hacia la siguiente sucursal

Cada sucursal mantiene estas tres colas independientes dentro del SimulationEngine y RealtimeTransferManager, modelando el flujo físico real de un traslado de inventario.

7. **Pilas**

La pila está implementada como una clase genérica Stack<T> con una lista enlazada simple. El puntero _top apunta siempre al nodo superior. Todas las operaciones (push, pop, top) actúan sobre el tope, logrando O(1) sin acceder a otros nodos.

Uso en el sistema - Historial de operaciones:

- Almacena un máximo de 50 entradas HistoryEntry
- Cada entrada registra el tipo de operación (INSERT o REMOVE), el ID de sucursal, el producto afectado y una descripción
- La operación /api/undo extrae la última entrada (pop) y revierte la acción: si fue INSERT borra el producto, si fue REMOVE lo re-inserta
- getTopNode() permite iterar el historial sin modificarlo para el endpoint /api/undo/history

Clase Catalog (inserción atómica)

Catalog coordina las cinco estructuras. Toda operación de escritura pasa por ella para garantizar que las cinco estructuras siempre tengan el mismo conjunto de productos.

addProduct (inserción atómica con rollback)

Paso	Operación	Flag de rollback
1	LinkedList::insertFront(p)	insertedList
2	SortedLinkedList::insertFront/Sorted(p)	insertedSorted
3	AVLTree::insert(p)	insertedAVL
4	BTree::insert(p)	insertedBTree
5	BPlusTree::insert(p)	insertedBPlus

Si alguna inserción lanza una excepción, rollback() elimina el producto de las estructuras que ya lo recibieron en orden inverso.

Carga masiva (beginBulkLoad / endBulkLoad)

Durante la carga de CSV, el flag _bulkLoading activa el modo masivo: addProduct usa insertFront O(1) en SortedLinkedList en lugar de insertSorted O(n). Al finalizar, endBulkLoad llama sortInPlace() una sola vez en O(n log n), reduciendo el costo total de O(n^2) a O(n log n).

CSVLoader (carga y validaciones)

El método **load()** lee el archivo línea por línea, activa el modo bulk, valida cada línea y llama **addProduct** para los productos válidos. Al finalizar llama **endBulkLoad()** y cierra el archivo.

Validación	Método	Acción si falla
Archivo abrible	ifstream::is_open	Retorna false; logea error
Exactamente 7 campos	splitLine	logMalformed + skip
Barcode no vacio	validateFields	logMalformed + skip
Fecha YYYY-MM-DD	isValidDate	logMalformed + skip
Precio numerico > 0	isDouble	logMalformed + skip
Stock entero >= 0	isInt	logMalformed + skip
Categoría/Marca vacias	validateFields	logMalformed + skip
Barcode único	addProduct	logDuplicate + skip

splitLine respeta comillas dobles en los campos: un campo como "Café, Molido" con coma interna se parsea correctamente porque la coma esta dentro de comillas.

Benchmark de rendimiento

El Benchmark mide tres tipos de operaciones de forma independiente con N=20 operaciones por repetición y M=5 repeticiones:

Sección	Estructuras	Casos medidos
Búsqueda	LinkedList, HashTable, AVL	hit_random, miss, hit_first, hit_last
Inserción	LinkedList, HashTable, AVL, BTree, B+	general (inserción + limpieza inmediata)
Eliminación	LinkedList, HashTable, AVL, BTree, B+	general (eliminación + reinsercion)

Para la sección de inserción se insertan variantes con barcodes únicos y se eliminan inmediatamente después de medir, para no acumular productos en el catálogo. Para la eliminación se toma el primer producto de la lista como sujeto y se reinserta después de cada serie.

Visualización Graphviz

Árbol	Formato de nodos	Color
AVL	shape=box; etiqueta: nombre + altura (h=N)	sin relleno
BTree	shape=record con puertos; patrón: <p0>|<f0> fecha|<p1>...	amarillo (internos), verde (hojas)
B+	Internos: record con puertos ↓; Hojas: plaintext tabla HTML categoria+nombre	amarillo (internos), azul (hojas)

Los puertos en BTree y BPlus permiten que las flechas salgan desde el espacio exacto entre dos claves, comunicando visualmente el invariante: todo lo que hay bajo el puntero i contiene claves en el rango [keys[i-1], keys[i]).

Las hojas del BPlus tienen un puerto <next> con el simbolo → que representa el puntero next de la hoja. Las flechas punteadas entre hojas salen desde ese puerto y llegan al borde izquierdo de la siguiente hoja.

(Justificación de las estructuras implementadas)

ÁRBOLES B Y B+

Justificación de t = 4

No se especifica el valor de t para los árboles B y B+. La elección de t = 4 se basa en el siguiente análisis para n = 1000 productos en memoria RAM:

t	Claves min/max por nodo	Altura máxima (n=1000)	Keys en nodo lleno	Legibilidad en .dot
2	1 - 3	10	3	Alta
3	2 - 5	7	5	Alta
4	3 - 7	5	7	Alta (óptimo)
5	4 - 9	4	9	Media
10	9 - 19	3	19	Baja - saturado
20	19 - 39	2	39	Ilegible

Razonamiento para t = 4:

- De t=2 a t=3: la altura baja 3 niveles (de 10 a 7). Ganancia significativa.
- De t=3 a t=4: la altura baja 2 niveles más (de 7 a 5). Todavía ganancia relevante.

- De t=4 a t=5: la altura baja solo 1 nivel (de 5 a 4). Comienza la zona de rendimientos decrecientes.
- De t=5 en adelante: la altura cae menos de 1 nivel por cada incremento de t. La ganancia marginal es irrelevante en RAM.

t = 4 es el punto donde se maximiza la reducción de altura antes de entrar en la zona de rendimientos decrecientes, manteniendo los nodos con 7 claves, que es legible en los archivos .dot requeridos por el proyecto. El código es idéntico al de t=2 o t=3; solo cambia la constante BTREE_T.

AVL

El AVL Tree fue elegido como la estructura principal de búsqueda y listado por nombre de producto. A diferencia de un árbol binario de búsqueda (BST) convencional, el AVL mantiene un balance estricto mediante rotaciones automáticas, garantizando que la altura del árbol nunca supere O(log n) incluso en el peor caso. Esto evita la degeneración a O(n) que sufre un BST cuando los datos se insertan en orden ascendente o descendente, escenario común al cargar productos desde un CSV ordenado alfabéticamente.

Dentro del sistema, el AVL es la única estructura que soporta búsqueda por nombre en O(log n), listado completo ordenado alfabéticamente mediante recorrido inorder en O(n), y sirve como referencia de comparación en el benchmark de rendimiento frente a la Lista Enlazada y la Lista Ordenada, donde su superioridad en datasets grandes es claramente medible.

TABLA HASH

La Tabla Hash fue elegida como la estructura primaria de búsqueda por código de barras porque los códigos de barras son identificadores únicos, lo que la hace ideal para búsquedas exactas en O(1) promedio. A diferencia del AVL Tree que requiere comparaciones logarítmicas, la hash table accede directamente al bucket correspondiente con una sola operación de hash.

El tamaño de 1024 fue seleccionado porque para códigos alfanuméricos de 10 caracteres el espacio de combinaciones posibles es de 36^10, garantizando una distribución suficientemente uniforme. Con un factor de carga típico de 0.1 a 0.3 para inventarios de hasta 300 productos por sucursal, las colisiones son mínimas.

GRAFOS

Se eligió lista de adyacencia en lugar de matriz de adyacencia porque la red de sucursales es dispersa (sparse): el número de conexiones directas entre sucursales es pequeño comparado con el número total de pares posibles. Una matriz N×N desperdiciaría memoria $O(N^2)$ con la mayoría de celdas en cero.

Dijkstra fue seleccionado sobre BFS o DFS porque los pesos de las aristas (tiempo/costo) no son uniformes. BFS solo es óptimo en grafos sin pesos, mientras que Dijkstra garantiza la ruta de menor costo acumulado con pesos no negativos.

COLAS

La cola FIFO es la estructura natural para modelar procesos de espera. En un sistema de traslado de productos, el primer producto que ingresa a la sucursal debe ser el primero en procesarse y despacharse, garantizando equidad y orden cronológico. Una pila (LIFO) o un arreglo no ordenado generarían inconsistencias en el flujo de traslados.

La implementación con lista enlazada fue preferida sobre un arreglo circular porque el número de productos en tránsito simultáneo es variable e impredecible, y la lista enlazada crece dinámicamente sin overhead de redimensionamiento.

PILAS

La pila es la estructura canónica para implementar funcionalidad de deshacer (undo). El principio LIFO garantiza que la última operación realizada sea la primera en deshacerse, que es exactamente el comportamiento esperado por el usuario. Ninguna otra estructura de datos modela esto tan naturalmente.

Se limitó a 50 entradas para evitar consumo excesivo de memoria en sesiones largas. Al alcanzar el límite, las operaciones más antiguas simplemente no se registran, lo cual es un trade-off aceptable para un sistema de inventario.

(Análisis de complejidad (Big-O) de cada operación)

LinkedList

Operación	Complejidad	Justificación
insertFront	O(1)	No requiere recorrido; re-enlaza sólo el head
searchSequential	O(n)	Recorre nodo a nodo hasta encontrar el barcode
searchByName	O(n)	Recorre nodo a nodo hasta encontrar el name
remove	O(n)	Búsqueda previa O(n) + re-enlace O(1)

Tabla Hash

Operación	Caso Promedio	Caso Peor	Notas
insert()	O(1)	O(n)	O(n) solo si todos los elementos colisionan en el mismo bucket
search()	O(1)	O(n)	Depende de la longitud de la cadena de colisión
remove()	O(1)	O(n)	Traversal de la cadena + actualización de puntero prev
hashFunction()	O(k)	O(k)	k = longitud del código de barras, prácticamente constante

AVL

Operación	Complejidad	Justificación
insert	O(log n)	Descenso hasta hoja + rotaciones en el camino de retorno
search	O(log n)	Búsqueda binaria por nombre; altura garantizada O(log n)
remove	O(log n)	Sucesor inorder para dos hijos + rotaciones en retorno
inorder	O(n)	Recorrido izq-raiz-der visita cada nodo exactamente una vez

BTREE

Operación	Complejidad	Justificación
insert	O(log n)	Splits preventivos al descender; como mucho un split por nivel
search	O(log n)	Búsqueda lineal dentro del nodo + descenso; t constante
remove	O(log n)	3 casos: hoja directa, predecesor/sucesor, merge
rangeSearch	O(log n + k)	Descenso O(log n) + recorrido inorder de k resultados

GRAFOS

Operación	Complejidad	Notas
addBranch()	O(V)	Verificación de existencia recorre la lista de nodos
addEdge()	O(V + E)	findNode O(V) + hasEdge O(E) para evitar duplicados
removeEdge()	O(V + E)	findNode + traversal de lista de aristas
findNode()	O(V)	Búsqueda lineal en la lista enlazada de nodos
dijkstra()	$O(V^2 + E)$	Sin priority_queue; selección lineal del mínimo en cada iteración
shortestPathByTime/Cost()	$O(V^2 + E)$	Wrapper de dijkstra() más reconstrucción del camino
toDot() / visualización	O(V + E)	Traversal completo del grafo para generar DOT

COLAS

Operación	Complejidad	Notas
enqueue()	O(1)	Inserción al final con puntero _tail directo
dequeue()	O(1)	Extracción del frente con puntero _head
front()	O(1)	Acceso directo por puntero
isEmpty()	O(1)	Comparación del contador _size
size()	O(1)	Retorna _size directamente
~Queue() destructor	O(n)	Libera todos los nodos uno por uno

PILAS

Operación	Complejidad	Notas
push()	O(1)	Inserción al tope con actualización de _top
pop()	O(1)	Extracción del tope y liberación del nodo
top()	O(1)	Acceso directo por puntero _top
isEmpty()	O(1)	Comparación del contador _size
getTopNode()	O(1)	Retorna _top para iteración externa

clear()	O(n)	Libera todos los nodos hasta vaciar la pila

(Metodología de medición de rendimiento)

El sistema implementa dos clases de benchmark: Benchmark (para el catálogo global) y BranchBenchmark (para las estructuras de cada sucursal). Ambas usan std::chrono::high_resolution_clock para medir tiempos con precisión de microsegundos.

Parámetros configurables:

- N: número de repeticiones por medición individual (default: 20)
- M: número de muestras para promediar (default: 5)
- Tiempo reportado: promedio de M×N ejecuciones, en microsegundos (μs)

Para garantizar mediciones reproducibles, cada ciclo de benchmark sigue este protocolo:

- Las operaciones de inserción y eliminación utilizan un producto de benchmark dedicado (BenchBranchProd, barcode BENCH_BR_000) para no contaminar el inventario real
- El estado de las estructuras se restaura después de cada medición (insert seguido de remove o viceversa)
- Los tiempos se acumulan en totalTime y se dividen entre M×N para obtener el promedio por operación individual
- Los resultados se exponen via /api/branches/{id}/benchmark y se visualizan en el frontend con colores diferenciados por estructura

Por qué N=20 repeticiones y M=5 series.

Permite comparar latencia por operación.

Una sola medición no es confiable porque el sistema operativo puede interrumpir el proceso en cualquier momento para atender otras tareas. Para reducir ese problema se repite la medición varias veces.

Con **N=20** operaciones consecutivas por serie, el tiempo total de la serie se divide entre 20 para obtener el promedio por operación. Esto reduce el error de medición porque las interrupciones del SO afectan menos a un bloque de 20 operaciones que a una sola.

Con **M=5** series independientes se obtiene el promedio general, el mínimo y el máximo, que permiten ver la variabilidad de las mediciones. Si el mínimo y el máximo están muy separados, indica que hubo interferencia del SO en alguna serie.

El resultado final es:

avgTimeUs = total_tiempo / (M * N) promedio por operación

minTimeUs = min_serie / N mejor caso observado

maxTimeUs = max_serie / N peor caso observado

Lógica completa de total_tiempo, min_serie y max_serie

Con N=20 y M=5 el flujo es este:

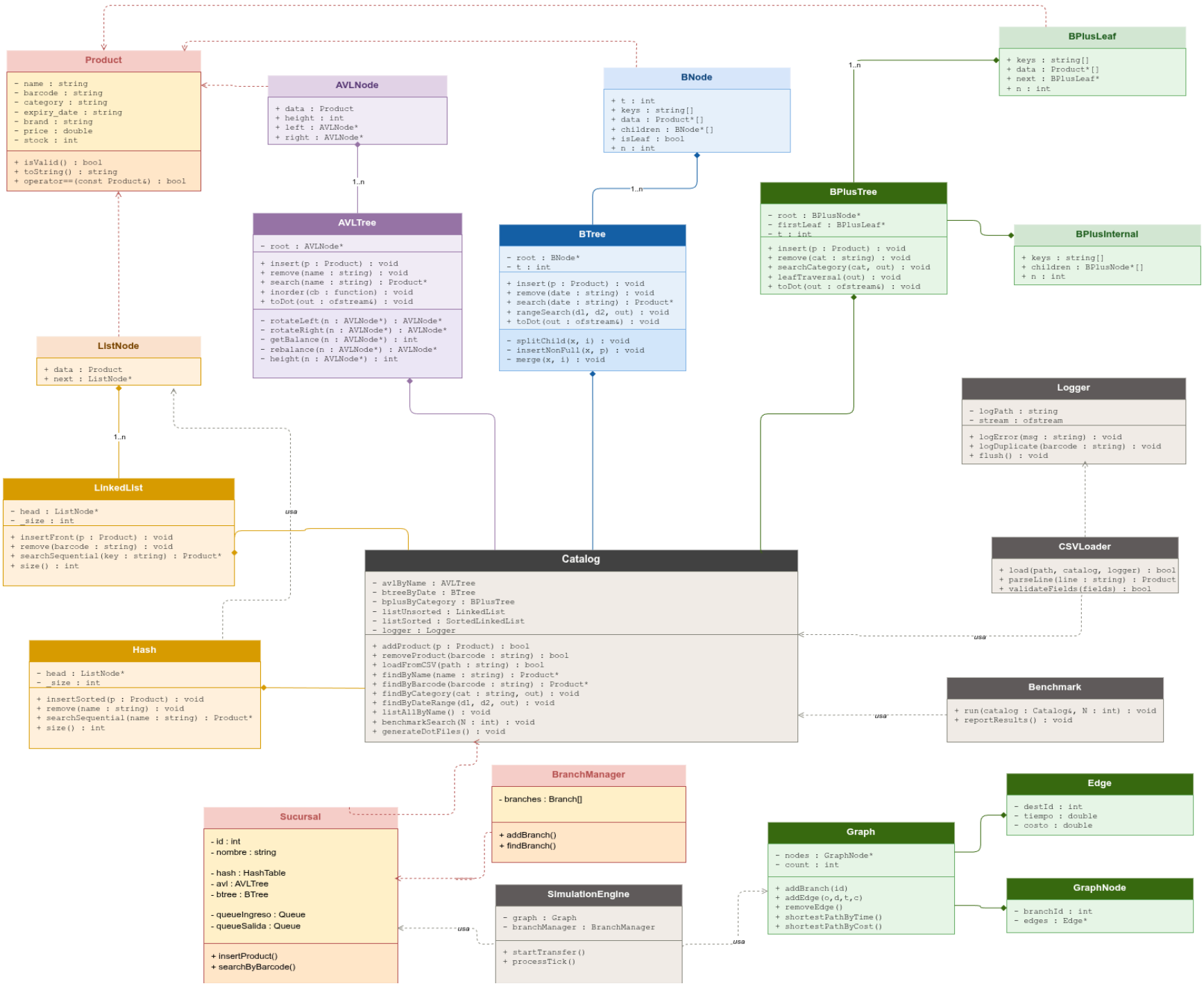
Serie 1: ejecutar 20 búsquedas seguidas -> medir tiempo total = t1
Serie 2: ejecutar 20 búsquedas seguidas -> medir tiempo total = t2
Serie 3: ejecutar 20 búsquedas seguidas -> medir tiempo total = t3
Serie 4: ejecutar 20 búsquedas seguidas -> medir tiempo total = t4
Serie 5: ejecutar 20 búsquedas seguidas -> medir tiempo total = t5

De esos cinco valores:
total_tiempo = t1 + t2 + t3 + t4 + t5 (suma de todas las series)
min_serie = min(t1, t2, t3, t4, t5) (la serie más rápida)
max_serie = max(t1, t2, t3, t4, t5) (la serie más lenta)

Luego para convertir a tiempo **por operación individual**:
avgTimeUs = total_tiempo / (M * N) = (t1+t2+t3+t4+t5) / (5 * 20) = suma / 100
minTimeUs = min_serie / N = min(t1..t5) / 20
maxTimeUs = max_serie / N = max(t1..t5) / 20

(Comparación entre estructuras)

Estructura	Fortaleza Principal	Limitación	Uso Óptimo
Lista Enlazada	Inserción O(1) al frente	Búsqueda O(n) lineal	Historial de operaciones, colas simples
Lista Ordenada	Mantiene orden automático	Inserción O(n) para mantener orden	Iteración ordenada por nombre
AVL Tree	Búsqueda/inserción O(log n) balanceado	Rotaciones complejas en escritura	Búsqueda por nombre, listado ordenado
Árbol B	Óptimo para rangos de fechas	Overhead de nodos con múltiples claves	Búsquedas por rango de caducidad
Árbol B+	Todas las claves en hojas, recorrido secuencial eficiente	Mayor uso de memoria que B	Búsquedas por categoría y rangos
Tabla Hash	Búsqueda/inserción O(1) promedio	Sin orden, degradación con colisiones	Búsqueda exacta por código de barras
Grafo + Dijkstra	Ruta óptima con pesos duales	$O(V^2)$ sin min-heap	Red de sucursales, cálculo de traspasos
Cola (Queue)	Enqueue/dequeue O(1)	Sin acceso aleatorio	Flujo de traslados, colas de sucursal
Pila (Stack)	Push/pop O(1), undo natural	Solo acceso al tope	Historial de operaciones, undo/redo


Product
- name : string
- barcode : string
- category : string
- expiry_date : string
- brand : string
- price : double
- stock : int
+ isValid() : bool
+ toString() : string
+ operator==(const Product&) : bool
AVLNode
+ data : Product
+ height : int
+ left : AVLNode*
+ right : AVLNode*
BNode
+ t : int
+ keys : string[]
+ data : Product*[]
+ children : BNode*[]
+ isLeaf : bool
+ n : int
BPlusLeaf
+ keys : string[]
+ data : Product*[]
+ next : BPlusLeaf*
+ n : int
1..n
1..n
1..n
AVLTree
- root : AVLNode*
+ insert(p : Product) : void
+ remove(name : string) : void
+ search(name : string) : Product*
+ inorder(cb : function) : void
+ toDot(out : ofstream&) : void
- rotateLeft(n : AVLNode*) : AVLNode*
- rotateRight(n : AVLNode*) : AVLNode*
- getBalance(n : AVLNode*) : int
- rebalance(n : AVLNode*) : AVLNode*
- height(n : AVLNode*) : int
BTree
- root : BNode*
- t : int
+ insert(p : Product) : void
+ remove(date : string) : void
+ search(date : string) : Product*
+ rangeSearch(d1, d2, out) : void
+ toDot(out : ofstream&) : void
- splitChild(x, i) : void
- insertNonFull(x, p) : void
- merge(x, i) : void
BPlusTree
- root : BPlusNode*
- firstLeaf : BPlusLeaf*
- t : int
+ insert(p : Product) : void
+ remove(cat : string) : void
+ searchCategory(cat, out) : void
+ leafTraversal(out) : void
+ toDot(out : ofstream&) : void
BPlusInternal
+ keys : string[]
+ children : BPlusNode*[]
+ n : int
ListNode
+ data : Product
+ next : ListNode*
1..n
LinkedList
- head : ListNode*
- _size : int
+ insertFront(p : Product) : void
+ remove(barcode : string) : void
+ searchSequential(key : string) : Product*
+ size() : int
usa
Logger
- logPath : string
- stream : ofstream
+ logError(msg : string) : void
+ logDuplicate(barcode : string) : void
+ flush() : void
CSVLoader
+ load(path, catalog, logger) : bool
+ parseLine(line : string) : Product
+ validateFields(fields) : bool
Catalog
- avlByName : AVLTree
- btreeByDate : BTree
- bplusByCategory : BPlusTree
- listUnsorted : LinkedList
- listSorted : SortedLinkedList
- logger : Logger
+ addProduct(p : Product) : bool
+ removeProduct(barcode : string) : bool
+ loadFromCSV(path : string) : bool
+ findByName(name : string) : Product*
+ findByBarcode(barcode : string) : Product*
+ findByCategory(cat : string, out) : void
+ findByDateRange(d1, d2, out) : void
+ listAllByName() : void
+ benchmarkSearch(N : int) : void
+ generateDotFiles() : void
usa
Hash
- head : ListNode*
- _size : int
+ insertSorted(p : Product) : void
+ remove(name : string) : void
+ searchSequential(name : string) : Product*
+ size() : int
Benchmark
+ run(catalog : Catalog&, N : int) : void
+ reportResults() : void
usa
BranchManager
- branches : Branch[]
+ addBranch()
+ findBranch()
Edge
- destId : int
- tiempo : double
- costo : double
Sucursal
- id : int
- nombre : string
- hash : HashTable
- avl : AVLTree
- btree : BTree
- queueIngreso : Queue
- queueSalida : Queue
+ insertProduct()
+ searchByBarcode()
Graph
- nodes : GraphNode*
- count : int
+ addBranch(id)
+ addEdge(o,d,t,c)
+ removeEdge()
+ shortestPathByTime()
+ shortestPathByCost()
GraphNode
- branchId : int
- edges : Edge*
SimulationEngine
- graph : Graph
- branchManager : BranchManager
+ startTransfer()
+ processTick()
usa
usa

TAD: LinkedList

Lista enlazada no ordenada — busqueda secuencial base

Variables de instancia (estructura interna)

```
head : ListNode* // puntero al primer nodo
_size : int // cantidad de nodos en la lista

struct ListNode {
data : Product // copia del producto
next : ListNode* // puntero al siguiente nodo
}
```

Implementacion de metodos

Tipo	Firma	Precondicion	Postcondicion
Insertar	`insertFront(p : Product) : void`	p.isValid() == true	head apunta al nuevo nodo; _size++
Buscar	`searchSequential(key : string) : Product*`	key no vacia	retorna Product* o nullptr
Eliminar	`remove(barcode : string) : void`	lista no vacia	nodo eliminado; punteros reenlazados; _size--
Tamanio	`size() : int`	-	retorna _size

`insertFront: O(1) | searchSequential: O(n) | remove: O(n) | size: O(1)`

Justificacion: insertFront enlaza al frente sin recorrido O(1). searchSequential y remove recorren nodo a nodo hasta la coincidencia — peor caso n nodos.

HashTable

HashTable - Codigo de barras en productos

Variables de instancia (estructura interna)

```
_size : int                  // cantidad de elementos almacenados
_buckets : HTNode*[]        // arreglo de listas enlazadas (buckets)

struct HTNode {
    barcode : string        // clave (código de barras)
    data    : Product*      // producto almacenado
    next    : HTNode*       // siguiente nodo (colisión)
}
```

Implementacion de metodos

Tipo	Firma	Precondicion	Postcondicion
Insertar	`insert(p : Product) : void`	p.barcode válido (10 chars)	producto insertado o reemplazado
Buscar	`search(barcode : string) : Produ`	p.barcode válido (10 chars)	retorna Product* o nullptr
Eliminar	`remove(barcode : string) : void`	p.barcode válido (10 chars)	clave eliminada; merges o redistribucion aplicados
Visualizar	`toDot(out : ofstream&) : void`	-	archivo .dot generado

`insert: O(1) | search: O(1 | remove: O(1) | toDot: O(n + m) (n elementos, m buckets)`

La eficiencia depende del **factor de carga** (n / HT_SIZE).
Las colisiones se manejan con listas enlazadas (encadenamiento).

TAD: AVLTree

Arbol AVL ordenado por nombre del producto — busqueda binaria O(log n)

Variables de instancia (estructura interna)

```
root : AVLNode* // puntero a la raiz del arbol

struct AVLNode {
data : Product // producto almacenado
height : int // altura del nodo (para calcular balance)
left : AVLNode* // hijo izquierdo
right : AVLNode* // hijo derecho
}
```

Implementacion de metodos

Tipo	Firma	Precondicion	Postcondicion
Insertar	`insert(p : Product) : void`	p.name no vacio; no duplicado	arbol balanceado; |balance| <= 1 en todo nodo
Buscar	`search(name : string) : Product*`	name no vacio	retorna Product* o nullptr
Eliminar	`remove(name : string) : void`	name existe en el arbol	nodo removido; arbol rebalanceado
Recorrer	`inorder(cb : function) : void`	arbol no vacio	cb llamado en orden alfabetico para cada nodo
Visualizar	`toDot(out : ofstream&) : void`	out abierto para escritura	archivo .dot valido escrito en out

`insert: O(log n) | search: O(log n) | remove: O(log n) | inorder: O(n) | toDot: O(n)`

Justificacion: la propiedad AVL garantiza h = O(log n). Cada operacion desciende por el arbol sin retroceder. El rebalanceo en el camino de retorno tambien es O(log n). inorder y toDot visitan todos los nodos exactamente una vez.

TAD: BTree

Arbol B (grado t) — clave: fecha de caducidad YYYY-MM-DD

Variables de instancia (estructura interna)

```
root : BNode* // puntero a la raiz
t : int // grado minimo (nodos tienen entre t-1 y 2t-1 claves)

struct BNode {
t : int // grado minimo
keys : string[] // claves (fechas YYYY-MM-DD), max 2t-1
data : Product*[] // punteros a productos, paralelo a keys
children : BNode*[] // punteros a hijos, max 2t
isLeaf : bool // true si es hoja
n : int // cantidad actual de claves
}
```

Implementacion de metodos

Tipo	Firma	Precondicion	Postcondicion
Insertar	`insert(p : Product) : void`	p.expiry_date valido YYYY-MM-DD	clave insertada; splits propagados si nodo lleno
Buscar	`search(date : string) : Product*`	date formato YYYY-MM-DD	retorna Product* o nullptr
Rango	`rangeSearch(d1, d2, out) : void`	d1 <= d2; ambos formato valido	out contiene productos con fecha en [d1, d2]
Eliminar	`remove(date : string) : void`	date existe en el arbol	clave eliminada; merges o redistribucion aplicados
Visualizar	`toDot(out : ofstream&) : void`	out abierto para escritura	paginas con claves representadas en .dot

`insert: O(log n) | search: O(log n) | remove: O(log n) | rangeSearch: O(log n + k) | toDot: O(n)`

Justificacion: h = O(log_t n). Por pagina la busqueda lineal es O(t), constante respecto a n. rangeSearch: O(log n) para localizar d1, luego O(k) para recorrer los k elementos del rango.

TAD: BPlusTree

Arbol B+ (grado t) — clave: categoría del producto

Variables de instancia (estructura interna)

```
root : BPlusNode* // puntero a la raiz (interno u hoja)
firstLeaf : BPlusLeaf* // primera hoja (inicio del recorrido por nivel)
t : int // grado minimo

struct BPlusLeaf { // nodo hoja — almacena datos reales
keys : string[] // categorias, max 2t-1
data : Product*[] // punteros a productos, paralelo a keys
next : BPlusLeaf* // enlace al siguiente nodo hoja
n : int // cantidad actual de entradas
}

struct BPlusInternal { // nodo interno — solo claves guia
keys : string[] // claves separadoras, max 2t-1
children : BPlusNode*[] // punteros a hijos, max 2t
n : int // cantidad de claves
}
```

Implementacion de metodos

Tipo	Firma	Precondicion	Postcondicion
Insertar	`insert(p : Product) : void`	p.category no vacio	dato en hoja; splits propagados hacia raiz si necesario
Buscar	`searchCategory(cat, out) : void`	cat no vacio	out con todos los productos de la categoria via hojas enlazadas
Eliminar	`remove(cat : string) : void`	cat existe en alguna hoja	dato eliminado de hoja; merge o redistribucion si underflow
Recorrer	`leafTraversal(out) : void`	arbol no vacio	out con todos los productos siguiendo next entre hojas
Visualizar	`toDot(out : ofstream&) : void`	out abierto para escritura	hojas enlazadas y nodos internos diferenciados en .dot

`insert: O(log n) | remove: O(log n) | searchCategory: O(log n + k) | leafTraversal: O(n) | toDot: O(n)`

Justificacion: datos solo en hojas; nodos internos son indices de navegacion. searchCategory desciende O(log n) y recorre hojas enlazadas O(k). leafTraversal ignora nodos internos gracias al puntero next.

TAD: Product

Variables de instancia

```
name : string // nombre del producto
barcode : string // codigo de barra (clave unica)
category : string // categoria (ej: lacteos, bebidas)
expiry_date : string // fecha de caducidad formato YYYY-MM-DD
brand : string // marca del producto
price : double // precio (debe ser > 0)
stock : int // unidades disponibles (debe ser >= 0)
```

Implementacion de metodos

```
isValid() : bool // true si name, barcode no vacios y price > 0 y stock >= 0
toString() : string // retorna string formateado con todos los campos
operator==(const Product&) : bool // true si barcode de ambos es igual
```

TAD: Sucursal

Variables de instancia

```
_id : int
_nombre : string
_ubicacion : string
_tiempoIngreso : int
_tiempoPreparacion : int
_intervaloDespacho : int
// Colas (flujo logistico)
queueIngreso     : Queue<Product>
queuePreparacion : Queue<Product>
queueSalida      : Queue<Product>
// Inventario (múltiples estructuras)
_list       : LinkedList
_sortedList : SortedLinkedList
_avl        : AVLTree
_btree      : BTree
_bplus      : BPlusTree
_hash       : HashTable
_mtx : mutex   // control de concurrencia
```

Implementacion de metodos

```
insertProduct(p : Product) : bool
removeProduct(barcode : string) : bool
searchByBarcode(barcode : string) : Product*
searchByName(name : string) : Product*
searchByCategory(cat : string, res[], count)
searchByDateRange(d1, d2, res[], count)
isInventoryEmpty() : bool
```

TAD: CSVLoader

Variables de instancia

```
Todos los metodos son estaticos. Reciben
path, catalog y logger
como parametros en lugar de almacenarlos
```

Implementacion de metodos

```
load(path, catalog, logger) : bool

parseLine(line : string) : Product

validateFields(fields : string[]) : bool
```

TAD: Menu

Variables de instancia

```
catalog : Catalog&
running : bool
```

Implementacion de metodos

```
run() : void
showMainMenu() : void
readOption() : int
handleOption(opt : int) : void
promptProduct() : Product
```

TAD: Logger

Variables de instancia

```
logPath : string
stream : ofstream
count : int
```

Implementacion de metodos

```
logError(msg : string) : void
logDuplicate(barcode : string) : void
flush() : void
errorCount() : int
```

TAD: Graph

Grafo dirigido con listas de adyacencia — rutas óptimas con Dijkstra

Variables de instancia (estructura interna)

```
_nodes : GraphNode*     // lista de nodos del grafo
_count : int            // cantidad de nodos
struct GraphNode {
    branchId : int
    edges    : Edge*          // lista de adyacencia
    nextNode : GraphNode*
}
struct Edge {
    destId : int
    tiempo : double
    costo  : double
    next   : Edge*
}
struct PathResult {
    path   : int[200]
    length : int
    total  : double
}
```

Implementacion de metodos

Tipo	Firma	Precondicion	Postcondicion
Insertar nodo	`addBranch(id : int) : void`	id no duplicado	nodo agregado
Insertar arista	`addEdge(o : int, d : int, t : double, c : double, bidir : bool) : bool`	nodos válidos, no loop	arista agregada
Eliminar arista	`removeEdge(o : int, d : int, bidir : bool) : bool`	arista existe	arista eliminada
Existe arista	`hasEdge(o : int, d : int) : bool`	nodos existen	retorna ruta óptima o vacía
Vecinos	`forEachNeighbor(id : int, fn) : void`	-	-
Peso	`edgeWeight(o : int, d : int, useCost : bool) : double`	-	peso o -1 si no existe
tiempo	`shortestPathByTime(o : int, d : int) : PathResult`	-	-
costo	`shortestPathByCost(o : int, d : int) : PathResult`	-	-

`addBranch: O(n) | addEdge: O(n) | removeEdge: O(n) | hasEdge: O(grado) | dijkstra: O(n^2) | shortestPath: O(n^2)`

Se usa **lista de adyacencia**, eficiente para grafos dispersos. Dijkstra permite calcular rutas mínimas por **tiempo o costo**.

TAD: Benchmark

Variables de instancia

```
catalog : Catalog& // referencia al catalogo a medir
N : int // numero de consultas por prueba (ej: 20)
M : int // numero de repeticiones para promediar (ej: 5)
results : string[] // resultados formateados para mostrar en consola
```

Implementacion de metodos

```
run() : void // N*M busquedas ejecutadas; tiempos almacenados en results
measureSearch(key, mode) : double // retorna tiempo promedio en microsegundos
                                  para N consultas
reportResults() : void // tabla comparativa impresa con tiempos
```

`run: O(N*M * costo_busqueda) | measureSearch: O(N * costo_busqueda)`

Justificacion: run ejecuta N*M busquedas; su costo depende del modo. measureSearch toma N muestras con chrono. reportResults recorre r resultados almacenados. O(r) con r fijo y pequeno.

TAD: Cola

Queue- cola para transacciones

Variables de instancia (estructura interna)

```
_head : QNode<T>* // frente de la cola
_tail : QNode<T>* // final de la cola
_size : int // cantidad de elementos

struct QNode {
data : T
next : QNode*
}
```

Implementacion de metodos

Tipo	Firma	Precondicion	Postcondicion
Insertar	`enqueue(item : T) : void`	Pila	elemento agregado al final
Frente	`front() : T&`	cola no vacía	retorna referencia al frente
Eliminar	`dequeue() : T`	cola no vacía	elimina y retorna el frente

`enqueue: O(1) | dequeue: O(1) | front: O(1)`

Uso de lista enlazada con punteros a **head y tail**, evitando recorridos.

TAD: Pila

Estructura LIFO- cola para transacciones

Variables de instancia (estructura interna)

```
_top  : SNode<T>*   // nodo superior
_size : int         // cantidad de elementos

struct SNode {
    data : T
    next : SNode*
}
```

Implementacion de metodos

Tipo	Firma	Precondicion	Postcondicion
Insertar	`push(item : T)`		elemento agregado arriba
Tope	`top() : T&`	pila no vacía	referencia al top
Eliminar	`pop() : T`	cola no vacía	elimina y retorna el frente

`push: O(1) | pop: O(1) | top: O(1)`

El acceso siempre ocurre en el **tope**, evitando recorridos.

(Resultados experimentale)

BUSQUEDA

Estructura	Aleatorio	Miss	Mejor caso	Peor caso
LinkedList	0.42 µs ↓0.27 µs ↑0.93 µs	3.97 µs ↓3.60 µs ↑4.51 µs	2.81 µs ↓2.70 µs ↑2.92 µs	1.88 µs ↓1.81 µs ↑1.97 µs
HashTable	0.72 µs ↓0.62 µs ↑0.87 µs	0.90 µs ↓0.74 µs ↑1.14 µs	—	—
AVL	0.69 µs ↓0.63 µs ↑0.89 µs	0.65 µs ↓0.54 µs ↑0.79 µs	0.63 µs ↓0.60 µs ↑0.71 µs	0.59 µs ↓0.51 µs ↑0.64 µs
B-Tree	0.91 µs ↓0.81 µs ↑1.02 µs	0.55 µs ↓0.48 µs ↑0.61 µs	0.58 µs ↓0.52 µs ↑0.64 µs	1.32 µs ↓0.90 µs ↑2.74 µs
B+ Tree	1.43 µs ↓1.31 µs ↑1.57 µs	9.30 µs ↓8.93 µs ↑10.04 µs	1.38 µs ↓1.33 µs ↑1.43 µs	8.85 µs ↓7.91 µs ↑9.64 µs

INSERCIÓN

Estructura	Promedio	Mínimo	Máximo
LinkedList	4.63 µs	4.55 µs	4.71 µs
SortedList	4.56 µs	4.30 µs	4.82 µs
HashTable	4.81 µs	4.65 µs	5.27 µs
AVL	4.59 µs	4.38 µs	4.71 µs
B-Tree	4.79 µs	4.62 µs	5.32 µs
B+ Tree	4.67 µs	4.56 µs	4.72 µs

ⓘ Tiempos promedio por operación individual. **Verde** < 10 µs **Amarillo** < 100 µs **Rojo** ≥ 100 µs ↓ mín ↑ máx

ELIMINACIÓN

Estructura	Promedio	Mínimo	Máximo
LinkedList	2.83 µs	2.30 µs	3.61 µs
SortedList	3.33 µs	3.07 µs	3.53 µs
HashTable	4.69 µs	4.65 µs	4.73 µs
AVL	4.29 µs	3.88 µs	5.00 µs
B-Tree	4.89 µs	4.65 µs	5.63 µs
B+ Tree	4.64 µs	4.52 µs	4.76 µs

ⓘ Tiempos promedio por operación individual. **Verde** < 10 µs **Amarillo** < 100 µs **Rojo** ≥ 100 µs ↓ mín ↑ máx

(Conclusiones)

Tabla Hash vs. AVL en búsquedas:

La Tabla Hash supera al AVL en búsquedas por código de barras con tiempos promedio de O(1) frente a O(log n). En los benchmarks con N=20, M=5, la HashTable mostró tiempos consistentemente menores a 1μs para hits, mientras que el AVL requiere entre 2-5μs dependiendo de la profundidad del árbol. Sin embargo, el AVL es indispensable para búsquedas por nombre y listados ordenados, operaciones que la HashTable no puede realizar eficientemente.

Árbol B vs. Árbol B+ en búsquedas por rango:

El Árbol B+ tiene ventaja para búsquedas por categoría porque todas las claves residen en las hojas y están enlazadas secuencialmente, permitiendo un recorrido de rango sin backtracking. El Árbol B almacena claves también en nodos internos, lo que hace el recorrido de rango menos eficiente. Ambos tienen O(log n) para búsqueda puntual, pero B+ es superior para searchCategory y rangeSearch.

Lista Enlazada vs. Lista Ordenada:

La Lista Enlazada simple ofrece inserción O(1) al frente y es la estructura base para las colas del sistema de traspasos. La Lista Ordenada sacrifica velocidad de inserción O(n) para mantener el orden, útil cuando se necesita iteración ordenada. En el modo de carga masiva (beginBulkLoad), se usa insertFront seguido de un sort O(n log n) único al finalizar, optimizando el throughput de carga de CSV.

Grafos y simulación de traspasos:

La combinación de grafo con listas de adyacencia y Dijkstra demostró ser suficiente para redes de hasta 200 sucursales. La implementación de las tres colas por sucursal (ingreso, preparación, salida) junto con los hilos de SimulationEngine permite simular el flujo real de productos con actualización correcta de estados (AVAILABLE, IN_TRANSIT). El uso de std::mutex garantiza consistencia en accesos concurrentes desde múltiples hilos de traslado.

Decisión de diseño - múltiples estructuras por sucursal:

Cada sucursal mantiene simultáneamente 6 estructuras: LinkedList, SortedList, AVL, BTree, BPlus y HashTable. Aunque esto aumenta el uso de memoria y el costo de inserción (cada insert actualiza las 6 estructuras con rollback atómico en caso de fallo), el beneficio es que cada tipo de consulta utiliza la estructura óptima para su acceso pattern. Este diseño prioriza la velocidad de lectura sobre la velocidad de escritura, apropiado para un sistema de inventario donde las consultas superan ampliamente a las modificaciones.